

Apuntes teorico 25/03

materia: programacion 4

profesor: Antonio

especificaciones que explicó al inicio sobre la materia:

- vamos a tener 2 parciales y 1 tp (mas el globalizador y el final)
 - se APRUEBA CON 4 en adelante.
 - info importante sobre el tp: engloba todo los contenidos, es importante metodología 2 para hacerlo, se puede hacer de grupos desde 3 hasta 5 integrantes
 - si se desaprueba el TP no vas a promocionar pero si puedes regularizar
 - otro punto a tener en cuenta, vamos a tener que elaborar una presentación para el tp.
-

cosas que vamos a hacer ahora:

repaso de contenidos de programacion 3

vamos a empezar con el front Y LUEGO pasamos full con el back end con REACT

es importante descargar ASP.NET

 ASP.NET Core, an open-source web development framework | .NET

y tenemos que tener SQL en las compus (OJO que las compus de la uni estan desactualizadas)

repaso de clase:

¿Qué es JSX?

JSX significa JavaScript XML (o JavaScript + XML).

Es una extensión de sintaxis de JavaScript que te permite escribir código que parece HTML directamente dentro de tus archivos de JavaScript.

¿Para qué sirve?

Sirve principalmente en **React** (la librería más popular para hacer páginas web interactivas) para:

- Describir cómo debe verse la interfaz de usuario (botones, textos, imágenes, etc.).
- Mezclar lógica de programación (variables, condicionales, bucles) con el diseño visual.
- Hacer que todo esté en un solo lugar (en un "componente"), lo que facilita mantener y actualizar el código.

En resumen: JSX hace que programar la parte visual de una app sea tan fácil como escribir HTML, pero con todo el poder de JavaScript.

HTML = lenguaje para hacer páginas estáticas.

JavaScript = lenguaje para hacer que las páginas hagan cosas (lógica).

JSX = la forma de mezclar los dos en React, para que sea fácil crear pantallas dinámicas.

Diferencia básica entre Expresión y Sentencia (Statement)

Concepto	Qué es	Devuelve un valor?	Ejemplos comunes	Se puede usar dentro de <code>{ }</code> en JSX?
Expresión	Código que produce un valor	Sí	<code>2 + 3</code> , <code>nombre</code> , <code>edad > 18</code> , <code>Math.random()</code>	Sí
Sentencia	Código que hace algo (una acción)	No (o no siempre)	<code>if (...)</code> , <code>for (...)</code> , <code>while</code> , <code>const x = ...</code> (declaración)	NO

Regla de oro en JSX:

- Dentro de las llaves `{ }` solo puedes poner expresiones.
- Nunca puedes poner sentencias completas como `if`, `for`, `while`, etc.

¿Por qué React/JSX solo permite expresiones?

Porque el JSX dentro de `return (...)` tiene que **generar un valor** (el elemento que se va a mostrar en pantalla).

Una expresión siempre devuelve algo que React puede usar. Una sentencia no devuelve nada útil para renderizar.

¿Qué son los Hooks?

Sirven principalmente para dos cosas grandes: **Gestionar estado** → recordar datos que cambian con el tiempo (como un contador, un formulario, un carrito de compras, etc.). **Hacer efectos secundarios** → ejecutar código en momentos específicos (cuando el componente se muestra, se actualiza o se elimina). Ejemplos: traer datos de una API, cambiar el título de la página, suscribirse a eventos, etc.

Los Hooks más importantes para principiantes

Hook	Para qué sirve	Ejemplo simple de uso
<code>useState</code>	Guardar y actualizar estado local	Contador, formulario, tema claro/oscuro
<code>useEffect</code>	Ejecutar código después del render (efectos)	Cargar datos de API, actualizar título
<code>useContext</code>	Acceder a datos globales sin pasar props	Tema de la app, usuario logueado

Las 3 reglas principales de los Hooks (muy importantes).

Estas reglas deben cumplirse siempre, si no React se rompe y aparecen errores raros. Solo llama a los Hooks en el nivel superior (Solo en el top level del componente) → NO los pongas dentro de if, for, while, funciones anidadas, ni después de un return temprano. Siempre al principio del componente, en el mismo orden.

Solo llama a los Hooks desde componentes de React → Solo dentro de componentes funcionales o dentro de custom hooks (hooks que tú creas y cuyo nombre empieza con use). NO los llames dentro de funciones normales de JavaScript, event handlers, callbacks, etc.

Los Hooks deben llamarse en el mismo orden en cada render,. React identifica cada hook por el orden en que aparecen. Si el orden cambia, React se confunde y pierde el estado.

EJERCICIO DE TATETI